

机器学出来的概念表达式可以只靠改写缩短 Syntactic Simplification of OWL Class Expressions

本篇范围声明：这是一篇 6 页的 arXiv 短论文 (2608.18899v1, 2026-08-19)。下面的「全文翻译」按章节推进，覆盖摘要与正文第 1 - 5 节的主体（引言、背景与化简原则、方法、评估、讨论与结论），两段算法伪代码保留结构、逐行说明其含义；文末 18 条参考文献列表不逐条翻译。

核心内容

Q：这篇解决的是什么问题？ A：用机器自动“学”出来的 OWL 概念定义，往往长得没法看。论文里最长的一条有 **4284 个构件单位**——这不是一个人写得出来的东西，是算法拼出来的。它长，人读不懂、维护不了；机器也吃亏，推理机要在这么大的表达式上跑，慢得多。这篇提出一个叫 ****CES (Class Expression Simplifier) **** 的算法，把这种表达式改短，**但不许改变它的含义**。

Q：「只改写不推理」是什么意思，为什么要这么限制自己？ A：化简一个逻辑表达式有两条路。一条是**语义化简**：调推理机去问「这两个写法是不是等价」，能发现更多机会，但你得先有一个推理机、还得有背景本体。另一条是**句法化简**：只认表达式自己的形状，套代数恒等式改写，比如「C 和 C 求交还是 C」。CES 走的是第二条。**代价是发现的机会少，好处是它不依赖推理机、不依赖背景本体，任何地方都能用**。论文自己把这个取舍写在结论里，说这是有意的设计选择。

Q：它套的是哪些规则？ A：都是布尔代数和描述逻辑里的老规矩，论文列了七类：**吸收律** ($C \sqcup (C \sqcap D) \equiv C$)、**幂等律** ($C \sqcap C \equiv C$)、**同一律与支配律** (和 $\top$ 、 $\perp$ 打交道的那几条)、**排中律与矛盾律** ($C \sqcup \neg C \equiv \top$)、**量词分配** (同一个角色上的多个 $\forall$ 合并成一个)、**基数约束包含** (同角色同填充的多个基数限制，按连接词取更松或更紧的那个)、**因式分解** ($(C \sqcap D) \sqcup (C \sqcap E) \equiv C \sqcap (D \sqcup E)$)。**没有一条是新发明的**——新的是把它们组织成一个能递归跑完、保证终止的单遍算法。

Q：算法结构上有什么值得注意的设计？ A：两点。一是 `simplify(c, p)` 带一个父节点参数 **p**——因为吸收律这类规则要看上一层是什么才能判，光看当前子表达式不够。二是因式分解前**先转成析取范式 (DNF)**，理由是把所有析取项摊到最外层，冗余项就是显式的顶层成分，不用在嵌套结构里反复遍历去找。

Q：效果如何，作者怎么证明“没改变含义”？ A：在两个中等规模的本体 (Carcinogenesis、Mutagenesis) 上取 200 条学出来的表达式测：**最高长度缩减 86%，最高推理耗时缩减 90%**，化简本身的平均耗时 0.11 - 0.16 秒，最长那条 4284 的表达式花了 1.34 秒。

证明方式这里要特别注意：作者**没有**给出形式化的正确性证明。原话是——单条改写规则各自保语义是清楚的，但要证明它们**任意组合**之后仍然正确，需要一个全局合流性 (confluence) 与终止性证明，**不在本文范围内**。他们改用**实证**：拿原表达式和化简后的表达式各跑一次实例检索，**比对返回的个体集合是否相同**。

Q：作者自己承认了哪些局限？ A：三条，都写在结论里。一、**样本单一**：200 条表达式全部由 TDL 这一个学习器产生，而作者明说选它正是因为它“以产出冗长假设著称”——换个更简洁的学习器，缩减幅度会小很多。二、**单遍算法对顺序敏感**：因式分解和基数合并谁先谁后会影响结果，最优顺序是开放问题。三、**句法层的天花板**：纯句法找不到需要推理才能发现的等价关系。

背景补充 / 点评

几个术语，先说清楚

OWL 与描述逻辑 (DLs) 的关系：OWL 是 W3C 的本体语言标准，它的形式基础是描述逻辑。论文开头就声明「本文中『概念』和『类表达式』两个词互换使用」，因为描述逻辑里的 concept 和 OWL 里的 class expression 是一回事，只是两套话语体系。

SR0IQ(D) 和 S0IQ(D) 是什么：这是描述逻辑给自己的表达能力分的档，每个字母代表一族构造子（R 是角色公理、O 是标称类、I 是逆角色、Q 是限定基数、D 是数据类型）。**SR0IQ(D)** 是 OWL 2 DL 对应的那一档。这篇把自己限制在 **S0IQ(D)** ——去掉了 R，只保留传递角色、排除其它所有 RBox 公理。为什么？因为 RBox 公理（比如“角色 A 是角色 B 的子角色”）**是本体级的知识，不在表达式自身里**，要用它就得读背景本体，那就不叫句法简化了。这个限制和“不依赖背景本体”这个卖点是同一件事的两面。

Class Expression Learning (CEL) 是干什么的：给一批正例个体和一批反例个体，让算法自动学出一个 OWL 概念定义，使得它恰好覆盖正例、排除反例。这是本体工程里的自动化路子——**人不写定义，机器从数据里归纳出定义**。这篇要收拾的烂摊子就是它产生的。

FaCT++ 是最接近的对照物：一个经典的 OWL 推理机，它内部会把表达式转成简化范式 (SNF)，但那是为推理服务的内部步骤，不以“给人看得懂的表达式”为目标。论文说它是“最接近但仍然有限”的对比点。

OWLAPY / Ontolearn：都是 Paderborn 大学同一个组的东西。OWLAPY 是 Python 的 OWL 本体工程框架，CES 就实现在里面；Ontolearn 是大规模 OWL 概念学习框架，产出这些冗长表达式的 TDL 学习器在它里面。**所以这是一个组自己造轮子、自己发现轮子产出太丑、再自己造个抛光机的完整链条**——这也解释了为什么样本全来自 TDL。

我的判断

这篇的价值不在算法，在它把“正确性怎么保证”这件事讲得很诚实。

那七条改写规则，任何学过离散数学的人都写得出来。真正的难点在**组合**：单遍递归里既要展平、又要吸收、又要合并基数、还要因式分解，规则之间会互相影响。作者面对这个难点的做法值得注意——**他没有硬凑一个证明，而是明确说“全局合流性证明不在本文范围内”，然后换了一条实证路径：比对两个表达式检索出的个体集合是否一致。**

这个选择我很认同，理由是：**语义等价的定义本身就是“在所有解释下外延相同”，那么在一个具体本体上比对外延，就是这个定义的直接抽样验证。** 它不是一个偷懒的替代品，是同一个定义的可执行版本。

但有一个假设，作者一笔带过了，我觉得分量不轻。

第 2.2 节最后一句：「我们的函数按照**唯一名称假设（UNA）**处理表达式。」

UNA 的意思是"不同的名字指不同的东西"。而标准的 OWL 语义恰恰不采用 UNA——在 OWL 里，`:张三` 和 `:老张` 在没有显式声明 `owl:differentFrom` 之前，可以是同一个个体。这不是细节，它直接影响基数约束那一类规则：`≥ 2 hasChild.Person` 在 UNA 下和在非 UNA 下，判定条件不一样。

论文没有讨论这个假设在什么场景下安全、什么场景下会出错。如果实现里真的按 UNA 化简了基数约束，而使用者的本体是标准 OWL 语义，就存在一类静默的语义偏移——而且实证验证抓不到它，因为验证用的 StructuralReasoner 是 OWLAPY 自带的结构推理机，它本身就不做完整的 OWL 语义推理。这是我读完最想去代码里确认的一件事。

另一处留白：全文只报了"最高缩减 86%"、"最高缩减 90%"，没有给平均值和分布。图 1 画了 100 条排序后的曲线，但正文里没有一个"平均缩减多少"的数字。最高值是最好看的那个数，用它做主要战报，说服力比平均值弱。

对我的启示

第一条，和我昨天读 data.world 那个基准得到的结论撞上了，而且是同一个道理的两个面。

那边的规矩是「评分只比执行结果，不比语句」——判两条 SQL 对不对，不看写法，看跑出来的数一不一样。这边是「判两个概念表达式等不等价，不证形式化，比检索出来的个体集合」。两处都是把"语法上说不清的等价"，换成"执行结果上可以直接对照的相等"。

这条我可以直接用在 prove 上。我们的语义层 DSL 编译出 SQL，如果哪天要做"两个指标定义是不是等价"这类判断（比如去重、比如检查一个新指标是不是已有指标的换写法），不要试图在 DSL 层面做符号比对，应该跑出来比结果。前者要处理无穷多种写法，后者只要一次执行。

第二条，关于"该不该在上层再造一个平行机制"。

这篇算法的位置很清楚：它是学习器的后处理步骤，作者说可以"无缝集成到概念学习系统里，作为呈现最终假设前的默认后处理"。它没有去改 TDL 让它别产出这么长的表达式，而是在下游加一道抛光。

这个形状值得警惕一下。产出冗长表达式的根因在学习器的搜索策略里，后处理只是把症状盖住了。好处是解耦——CES 对任何学习器都能用；代价是学习器那边永远没有改进压力，而且每多一个消费者就得记得挂上这道后处理。这篇没有讨论这个取舍。

放到我自己身上：prove 的指标注册表里那五个具名过滤器被 use 了 10 次、9 次、7 次，那也是同一个形状——真正的事实（"完成度表永远排除已删除行"）属于实体，却被复制进了每个用途。我当时的判断是应该把它移回实体定义那一侧。这篇论文提供的是反例视角：有时候在下游做统一后处理，确实比改上游更省、适用面更广。判据是——这份知识是"关于事实的"还是"关于呈现的"。表达式冗长是呈现问题，放下游对；"要排除已删除行"是事实问题，放下游错。这个区分我以前没想清楚。

第三条，是个具体可抄的工程细节。

`simplify(c, p)` 把父节点当参数传下去，因为吸收律需要跨层的上下文。这是个很小的设计，但它解决的问题很常见：递归处理树形结构时，有一类规则光看当前节点判不了。常见的错误做法是先跑一遍收集全局信息、再跑一遍应用规则（两遍），或者用可变的全局状态。传父节点是最轻的那个解法。我们的 DSL 编译器里 `scope` / `attach` / `compose` 的嵌套处理，如果哪天遇到“这一层要不要展开取决于上一层是什么”的规则，这就是现成的做法。

文中金句

"While each individual rewrite rule used by the algorithm is semantics-preserving, proving correctness for their arbitrary composition would require a global confluence and termination proof, which is outside the scope of this work. Consequently, we evaluate correctness empirically via reasoning-based validation by comparing the sets of instances retrieved for the original and the simplified class expressions and confirming their equivalence." 虽然算法用到的每一条改写规则单独看都是保语义的，但要证明它们任意组合之后的正确性，需要一个全局合流性与终止性证明，这不在本文范围内。因此我们改用实证方式验证正确性：通过基于推理的验证手段，比对原始表达式与化简后表达式各自检索出的个体集合，确认二者等价。

"At present, CES operates exclusively at the syntactic level. This design choice avoids the need to rely on a reasoner, making the method broadly applicable and independent of the presence of a background ontology." 目前 CES 只在句法层面工作。这个设计选择使它不必依赖推理机，从而具有广泛的适用性，并且不依赖于背景本体是否存在。

"Our experiments deliberately focused on the TDL, which is known to produce lengthy and complex class expressions. In this setting, the value of our simplifier is particularly evident. For other learners that may generate more compact hypotheses, the gain in simplification may be less pronounced." 我们的实验刻意聚焦于 TDL，它以产出冗长复杂的类表达式著称。在这种设定下，我们化简器的价值格外明显。对于那些可能生成更紧凑假设的学习器，化简带来的收益或许没有这么显著。

"Nevertheless, the algorithm consistently provides a non-intrusive 'bonus' improvement in complexity reduction and reasoning efficiency, since it does not affect the semantics of the resulting expressions." 尽管如此，由于该算法不影响结果表达式的语义，它始终能提供一种非侵入式的“额外”收益：降低复杂度、提升推理效率。

全文翻译

摘要

Class Expression Learning (CEL) often produces complex OWL class expressions that are difficult to interpret and reason over. However, by following theoretically grounded simplification principles, this complexity can be reduced. In this paper, we propose Class Expression Simplifier (CES), a novel algorithm for the syntactic simplification of class expressions in Description Logics (DLs). CES aims to preserve formal semantics while reducing representational complexity.

类表达式学习 (CEL) 常常产出复杂的 OWL class expression, 这些表达式既难以解释也难以在其上做推理。不过, 只要遵循有理论依据的化简原则, 这种复杂度是可以被降低的。本文提出 [Class Expression Simplifier \(CES\)](#), 一个用于对描述逻辑 (DLs) 中的类表达式做句法化简的新算法。CES 的目标是在降低表示复杂度的同时保持形式语义不变。

It systematically applies rewriting rules to eliminate redundancies and identify simpler yet equivalent expressions, thereby producing more compact and human-readable representations without altering logical entailments. We evaluate the effectiveness of CES on class expressions learned from two medium-sized ontologies, demonstrating measurable improvements in reasoning efficiency and reductions in verbosity.

它系统性地应用改写规则来消除冗余、找出更简单但等价的表达式, 从而在不改变逻辑蕴含的前提下, 产生更紧凑、更适合人读的表达。我们在两个中等规模本体上学出来的类表达式上评估了 CES 的有效性, 展示了推理效率上可度量的提升和冗长程度的下降。

This work contributes to the broader goal of making ontology-driven applications more accessible, maintainable, and scalable, with direct implications for knowledge graph construction, semantic search, and Web-scale reasoning. CES is implemented within the open-source Python framework OWLAPY and is publicly available.

这项工作服务于一个更大的目标: 让本体驱动的应用更易用、更易维护、更可扩展; 它对 knowledge graph 构建、语义搜索和 Web 规模推理都有直接影响。CES 实现在开源 Python 框架 OWLAPY 之中, 且已公开可用。

关键词: 句法化简、OWL 类表达式、描述逻辑、基于规则的改写、语义网。

一、引言: 表达能力带来的代价

Ontologies expressed in the Web Ontology Language (OWL) have become a cornerstone for representing structured knowledge on the Semantic Web, powering applications ranging from biomedical data integration to knowledge graph construction and Web-scale reasoning. At the heart of OWL are class expressions, which allow the specification of rich and precise concepts by combining logical operators and role restrictions. While this expressiveness (e.g. SROIQ(D)) is essential for capturing domain semantics, it often leads to complex and verbose expressions.

用 Web Ontology Language (OWL) 表达的本体，已经成为在语义网上表示结构化知识的基石，支撑着从生物医学数据集成到 knowledge graph 构建、再到 Web 规模推理的各类应用。OWL 的核心是 class expression，它允许通过组合逻辑运算符和角色限制来刻画丰富而精确的概念。这种表达能力（例如 **SR0IQ(D)** 这一档）对于捕捉领域语义是必需的，但它也常常导致复杂而冗长的表达式。

Such expressions are not only difficult for ontology engineers to interpret and maintain, but can also introduce inefficiencies in automated reasoning and query answering tasks.

这样的表达式不仅让本体工程师难以解释和维护，还会给自动推理和查询应答任务带来效率损失。

The problem of complex class expressions manifests in several ways. From a usability perspective, verbose or redundant expressions reduce the readability of ontologies, making collaborative development, reuse and verbalization (e.g. via LLMs) challenging. From a computational perspective, the presence of unnecessary syntactic constructs can increase the size of reasoning tasks, negatively affecting the performance of DLs reasoners.

复杂类表达式带来的问题体现在几个方面。**从可用性角度**看，冗长或冗余的表达式降低了本体的可读性，使协同开发、复用以及口语化转述（例如借助 LLM）都变得困难。**从计算角度看**，不必要的句法构造会增大推理任务的规模，损害描述逻辑推理机的性能。

Simplifying class expressions without altering their semantics is therefore a desirable step to improve both the human- and machine-oriented aspects of ontology use. In Web-scale applications, where ontologies are combined with massive knowledge graphs, even small inefficiencies in reasoning can become prohibitive.

因此，在不改变语义的前提下化简类表达式，对本体使用中面向人和面向机器这两个方面都是一步有价值的改进。在 Web 规模的应用里，本体要和海量 knowledge graph 结合，推理上哪怕很小的低效也可能变得无法承受。

Existing work has primarily focused on semantic reasoning, optimization techniques, or ontology modularization. By contrast, systematic methods for the syntactic simplification of OWL class expressions remain largely underexplored. While FaCT++ provides basic simplification like for example, transforming expressions into a simplified normal form (SNF), it represents the closest-yet still limited-point of comparison to our approach.

已有工作主要集中在语义推理、优化技术或本体模块化上。相比之下，**针对 OWL 类表达式做句法化简的系统性方法，仍然基本没有被探索过**。FaCT++ 提供了一些基础的化简，例如把表达式转换成简化范式（SNF），它是与我们方法最接近、但仍然有限的对比点。

贡献。 本文通过引入一种植根于 **S0IQ(D)**（限制在传递角色，排除其它所有 RBox 公理）的句法改写方法，来应对类表达式化简这一挑战——之所以这样限定，是因为我们严格遵守“只做句法化简”这一原则。具体而言，我们：

- **(C1)** 定义了一组基于描述逻辑的改写规则，用于在限制到传递角色的 **S0IQ(D)** 上对概念做句法化简。

- (C2) 提出 CES：一个基于改写的类表达式句法化简算法，采用递归方式并保证终止。
- (C3) 在两个中等规模数据集上生成的 200 条类表达式上，展示了 CES 在缩短长度与降低推理耗时上的有效性。

二、背景：描述逻辑与七条化简原则

OWL ontologies are grounded in DLs, where complex class expressions are constructed using boolean connectives, quantifiers, and role restrictions. Throughout this paper we use the terms 'concept' and 'class expression' interchangeably because DLs concepts can be represented in OWL class expressions and vice-versa.

OWL 本体建立在描述逻辑之上，其中复杂的类表达式由布尔连接词、量词和角色限制构造而成。本文全文中我们把"概念"和"类表达式"两个词互换使用，因为描述逻辑中的概念可以表示为 OWL 类表达式，反之亦然。

DLs provide the formal foundation of OWL and define concepts using constructors such as conjunction ($\sqcap$), disjunction ($\sqcup$), negation ($\neg$), existential ($\exists r.C$), and universal ($\forall r.C$) restrictions over roles r . These constructs enable the definition of complex concepts that describe sets of individuals satisfying specific conditions. Formally, each DL expression is interpreted under a model $I = (\Delta I, \cdot I)$, where ΔI is the domain and $\cdot I$ an interpretation function mapping concepts and roles to subsets of ΔI and binary relations, respectively.

描述逻辑为 OWL 提供形式基础，用这样一些构造子来定义概念：合取 ($\sqcap$)、析取 ($\sqcup$)、否定 ($\neg$)，以及在角色 r 上的存在限制 ($\exists r.C$) 和全称限制 ($\forall r.C$)。这些构造使我们能够定义描述"满足特定条件的个体集合"的复杂概念。形式上，每个描述逻辑表达式都在一个模型 $I = (\Delta I, \cdot I)$ 下被解释，其中 ΔI 是论域， $\cdot I$ 是解释函数，分别把概念和角色映射到 ΔI 的子集和二元关系上。

化简原则。 面向句法化简这个目标，我们提出以下原则：

- **冗余消除 (Redundancy Elimination)**：删除子表达式的重复项，或对合取项、析取项做排序与展平。
- **等价保持 (Equivalence Preservation)**：应用代数定律或规则，产出更短但语义完全相同的表达式。

These principles guide the design of our rewriting algorithm, ensuring termination and semantic soundness. Below, we describe some of the most profound rules applied to simplify class expressions under the equivalence preservation principle. Our function processes expressions according to the unique name assumption (UNA).

这些原则指导了我们改写算法的设计，保证了终止性和语义可靠性。下面我们描述在等价保持原则下用于化简类表达式的几条最重要的规则。[我们的函数按照唯一名称假设 \(UNA\) 处理表达式。](#)

吸收律 (Absorption)：吸收律通过利用逻辑包含关系，消除冗余的析取或合取构造。形式上，对任意类表达式 C 和 D ，下列等价式成立： $C \sqcup (C \sqcap D) \equiv C$ 且 $C \sqcap (C \sqcup D) \equiv C$ 。

幂等律 (Idempotence): 幂等律保证同一个类表达式在合取或析取中重复出现, 不会增加它的含义: $C \sqcap C \equiv C$ 且 $C \sqcup C \equiv C$ 。

同一律与支配律 (Identity and Domination): 这两条定律描述类表达式与全称概念 $\top$ 和空概念 $\perp$ 的相互作用。同一律刻画中性行为, 支配律表达压倒性行为: $C \sqcap \top \equiv C$, $C \sqcup \perp \equiv C$, $C \sqcup \top \equiv \top$, $C \sqcap \perp \equiv \perp$ 。

排中律与矛盾律: 排中律表达"对任意类表达式 C , C 或它的否定必有其一普遍成立"; 矛盾律表达"一个类表达式和它的否定不可能同时被满足"。分别是: $C \sqcup \neg C \equiv \top$ 且 $C \sqcap \neg C \equiv \perp$ 。

量词分配 (Quantifier Distribution): 量词分配把同一个角色 r 上的多个全称限制或多个存在限制, 合并成一个带有合并后填充表达式的单一限制: $\forall r. C \sqcap \forall r. D \equiv \forall r. (C \sqcap D)$ 且 $\exists r. C \sqcup \exists r. D \equiv \exists r. (C \sqcup D)$ 。

基数限制包含 (Cardinality Restriction Subsumption): 当多个基数限制共享同一个角色和同一个填充概念时, 可以根据连接词选取更松或更紧的界, 从而消去冗余约束。形式上, 对角色 r 、类表达式 C 和满足 $m < n$ 的整数 m 、 n : $(\geq m \ r. C) \sqcup (\geq n \ r. C) \equiv (\geq m \ r. C)$ 且 $(\geq m \ r. C) \sqcap (\geq n \ r. C) \equiv (\geq n \ r. C)$ 。对上界限制 ($\leq$) 以及数据类型限制, 也使用类似的等价式。

因式分解 (Factorization): 因式分解利用分配律, 从析取或合取中提取公共子表达式, 减少重复和结构复杂度: $(C \sqcap D) \sqcup (C \sqcap E) \equiv C \sqcap (D \sqcup E)$ 且 $(C \sqcup D) \sqcap (C \sqcup E) \equiv C \sqcup (D \sqcap E)$ 。因式分解同样应用于被否定的概念, 把否定提取出来, 以减少表示这个复杂概念所需的否定构造的数量。

三、方法：一个带父节点参数的递归函数

We define syntactic simplification as a transformation function f on the class expression C such that $f(C)$ is syntactically simpler than C in terms of length - i.e. OWL constructs required to represent the expression, and semantically equivalent, i.e., $CI = f(C)I$ for all interpretations I .

我们把**句法化简**定义为作用在类表达式 C 上的一个变换函数 f , 使得 $f(C)$ 在长度意义上 (即表示该表达式所需的 OWL 构造子数量) 比 C 更简单, 并且**语义等价**, 即对所有解释 I 都有 $CI = f(C)I$ 。

When handling complex class expressions, we often encounter combinations of n -ary boolean connectives. For brevity, we refer to these simply as n -ary expressions and to the concepts in such expressions as operands. Our simplification algorithm is particularly effective in this setting, since a larger number of simplification rules can typically be applied. In contrast, when expressions lack such connectives, the simplification process is naturally less extensive.

在处理复杂类表达式时, 我们经常遇到 n 元布尔连接词 (即概念的合取与析取) 的组合。为简洁起见, 我们把它们直接称为 **n 元表达式**, 把其中的概念称为**操作数**。我们的化简算法在这种场景下格外有效, 因为通常可以应用更多条化简规则。反之, 当表达式里没有这类连接词时, 化简过程自然就没那么充分。

The objective of the algorithm is to apply as many rules within a single simplification pass as possible. To this end, we implemented a recursive function $\text{simplify}(c, p)$, where c denotes the

current class expression and p denotes its parent n -ary expression. The second parameter is essential for rules that require knowledge of the surrounding context, i.e., the class expression that is one level higher in the recursion. The `simplify` function is implemented as a single-dispatch function, with different specializations for each type of class expression, which makes recursive traversal both natural and efficient.

算法的目标是**在单遍化简中尽可能多地应用规则**（即在给定类表达式上执行算法一次）。为此我们实现了递归函数 `simplify(c, p)`，其中 c 表示当前类表达式， p 表示它的父 n 元表达式。**第二个参数对那些需要了解周围上下文的规则是必需的**——所谓上下文，就是递归中高一层的那个类表达式。`simplify` 函数以单派（single-dispatch）方式实现，针对每种类表达式类型有各自的特化版本，这使得递归遍历既自然又高效。

3.1 n 元布尔表达式的化简

For n -ary expressions, simplification proceeds both at the operand level and at the connective level. Nested n -ary expressions are first flattened to facilitate rule application, and then the Law of Idempotence is applied to remove repeated operands. During recursion, Absorption Law is applied when common elements exist between an expression and its parent, and duplicates are removed using set-based storage.

对 n 元表达式，化简同时在**操作数层面**和**连接词层面**进行。嵌套的 n 元表达式首先被展平（例如 $(C \sqcap (D \sqcap E))$ 变成 $(C \sqcap D \sqcap E)$ ），以便于规则应用；然后应用幂等律去掉重复的操作数。在递归过程中，当一个表达式与其父表达式之间存在公共元素时，应用吸收律；重复项则通过基于集合的存储来消除。

In case the $\top$ or $\perp$ concepts occur in the set of operands we apply the Law of Domination/Identity. The algorithm also applies the Law of the Excluded Middle, removing operands that appear alongside their negations, and handles subsumption checks for cardinality restrictions. Finally, factorization is applied via `apply_factorization` to reduce structural length.

如果操作数集合中出现 $\top$ 或 $\perp$ 概念，就应用支配律/同一律。算法还应用排中律，移除那些与自身否定同时出现的操作数，并处理基数限制的包含关系检查。最后，通过 `apply_factorization` 应用因式分解来缩短结构长度。

The `simplify` function for intersection (`ObjectIntersectionOf`) follows a similar procedure, with slight differences corresponding to the specific rules applicable to conjunctions.

针对交集（`ObjectIntersectionOf`）的 `simplify` 函数遵循类似流程，只在对应合取的那些特定规则上略有差异。

算法 1（针对 `ObjectUnionOf` 的递归函数 `simplify(c, p)`）的流程是：先调 `combine_nary_expressions(c)` 展平嵌套并联合应用幂等律，若表达式变了就带新值重新递归；取出操作数集合；若父节点不为空则应用吸收律移除与父节点的公共操作数；对每个操作数递归调用 `simplify`，得到集合 S ；若 S 只剩一个元素直接返回它；若 $\top \in S$ 则按支配律返回 $\top$ ；若 $\perp \in S$ 则按同一律把 $\perp$ 从 S 中删掉；遍历 S 的副本，若某元素是 `ObjectComplementOf` 且它的操作数也在 S 中，则按排中律把这

一对同时删掉；若 S 空了返回 $\top$ ，只剩一个则返回它；用 S 重建 `ObjectUnionOf`；把同角色同填充的基数限制合并；最后，若当前是根表达式 (p 为空)，才调用 `apply_factorization`。

3.2 因式分解

Factorization is important when it comes to reducing length of the expressions. Throughout the simplification process, intermediate class expressions may remain structurally verbose, even after the application of local rewriting rules and can further benefit from factorization which we apply at the root level of recursive union or intersection simplifications, following the simplification of all operands.

在缩短表达式长度这件事上，因式分解很重要。在整个化简过程中，中间类表达式即便已经应用了局部改写规则，结构上仍可能是冗长的，还能从因式分解中进一步获益——[我们在递归的并集或交集化简的根层面、在所有操作数都化简完之后，才应用因式分解](#)。我们认为这是重要的一步，因为因式分解可以大幅减少表示一个概念所需的 OWL 构造子数量。

Factorization is implemented recursively via the recursive function `apply_factorization(c, first_iteration)`. On the first call from the `simplify` function, the class expression c is converted into its top-level disjunctive normal form (DNF). This ensures that all disjunctions are explicitly represented at the outermost level, facilitating the identification of redundant or common subexpressions and reducing repeated traversal of nested structures. Furthermore, in DNF, redundant disjuncts are easier to identify because they appear as explicit top-level components rather than being hidden inside nested structures.

因式分解通过递归函数 `apply_factorization(c, first_iteration)` 实现。在从 `simplify` 函数发起的第一次调用中，类表达式 c 被转换成它的顶层析取范式 (DNF)。这确保所有析取都被显式地表示在最外层，便于识别冗余或公共的子表达式，并减少对嵌套结构的重复遍历。此外，在 DNF 下，冗余的析取项更容易被识别，因为它们是显式的顶层成分，而不是藏在嵌套结构里面。

To handle both unions and intersections uniformly, two dynamic variables, `type_a` and `type_b` are assigned the types `ObjectUnionOf` and `ObjectIntersectionOf` where `type_a` is equal to the type of the class expression c and `type_b` is the remaining one.

为了统一处理并集和交集，两个动态变量 `type_a` 和 `type_b` 被赋值为 `ObjectUnionOf` 和 `ObjectIntersectionOf` 这两个类型，其中 `type_a` 等于类表达式 c 自身的类型，`type_b` 是剩下的那个。

算法 2 的流程是：若是第一次迭代，先把 c 转成顶层 DNF；根据 c 确定 `type_a` 和 `type_b`；取出操作数集合，把它划分成“属于 `type_b` 类型的”和“其余的”两部分；若两部分都非空，在两个集合之间应用吸收律；若 `type_b` 那部分有 2 个及以上操作数，则**两两成对**地识别公共操作数、做局部因式分解，再把结果与剩余操作数合并；若只有 1 个，则递归地对这单个操作数做因式分解，再与其余部分组合；返回 c 。

This recursive design ensures that factorization proceeds from general to specific cases, repeatedly applying simplification until no further factorization is possible. The final result is a

structurally compact class expression that preserves the semantics of the original expression.

这种递归设计保证因式分解从一般情形推进到特殊情形，反复应用化简直到再也无法进一步分解为止。最终结果是一个结构紧凑、且保持了原表达式语义的类表达式。

四、评估：200 条表达式，最高缩减 86%

To assess the impact of our syntactic simplification procedure in terms of length reduction and reasoning efficiency, we generated 200 class expressions given random learning problems over two medium-sized datasets, Carcinogenesis and Mutagenesis, from the SML-bench suite. Complex and lengthy class expressions are generated with the Tree-based OWL Class Expression Learner (TDL) from the Ontolearn framework, which is known to produce significantly more verbose hypotheses compared to other concept learning algorithms on the framework.

为了评估我们的句法化简过程在缩短长度和提升推理效率上的作用，我们在 SML-bench 测试集里的两个中等规模数据集 **Carcinogenesis** 和 **Mutagenesis** 上，针对随机学习问题生成了 **200 条类表达式**。这些复杂而冗长的类表达式由 Ontolearn 框架中的**基于树的 OWL 类表达式学习器 (TDL)**生成——在该框架的各个概念学习算法里，TDL 以产出明显更冗长的假设著称。

We refer to the length of a given class expression as the number of the OWL constructs required to represent it. For reasoning efficiency we measure the runtime of instance retrieval using StructuralReasoner - the native reasoner of OWLAPY.

我们把一条类表达式的**长度**定义为表示它所需的 OWL 构造子数量。推理效率方面，我们用 OWLAPY 自带的推理机 **StructuralReasoner** 测量实例检索的运行时间。

While each individual rewrite rule used by the algorithm is semantics-preserving, proving correctness for their arbitrary composition would require a global confluence and termination proof, which is outside the scope of this work. Consequently, we evaluate correctness empirically via reasoning-based validation by comparing the sets of instances retrieved for the original and the simplified class expressions and confirming their equivalence.

虽然算法用到的每一条改写规则单独看都是保语义的，**但要证明它们任意组合之后的正确性，需要一个全局合流性与终止性证明，这不在本文范围内**。因此我们改用实证方式验证正确性：通过基于推理的验证手段，比对原始表达式与化简后表达式各自检索出的个体集合，确认二者等价。

We recorded the highest length reduction to be 86% less than the original expression and the highest runtime reduction to be 90% less than the original. For the conducted experiments the average runtime of simplification is 0.11s on Mutagenesis and 0.16s on Carcinogenesis with the longest expression of length 4284 taking 1.34 seconds to simplify.

我们记录到的**最高长度缩减为原表达式的 86%，最高运行时缩减为原来的 90%**。在所做的实验中，化简本身的平均耗时在 Mutagenesis 上是 0.11 秒、在 Carcinogenesis 上是 0.16 秒；最长的那条表达式长度为 **4284**，化简它花了 1.34 秒。

Our experiments deliberately focused on the TDL, which is known to produce lengthy and complex class expressions. In this setting, the value of our simplifier is particularly evident. For other learners that may generate more compact hypotheses, the gain in simplification may be less pronounced. Nevertheless, the algorithm consistently provides a non-intrusive "bonus" improvement in complexity reduction and reasoning efficiency, since it does not affect the semantics of the resulting expressions.

我们的实验刻意聚焦于 TDL，它以产出冗长复杂的类表达式著称。在这种设定下，我们化简器的价值格外明显。对于那些可能生成更紧凑假设的学习器，化简带来的收益或许没有这么显著。尽管如此，由于该算法不影响结果表达式的语义，它始终能提供一种非侵入式的"额外"收益：降低复杂度、提升推理效率。

五、讨论与结论：三条局限，以及往语义层走的下一步

In this work, we introduced CES, a syntactic simplification algorithm for OWL class expressions, and showed its effectiveness in reducing verbosity and improving reasoning efficiency. We highlight the fact that the method can be seamlessly integrated into class expression learning systems (e.g. with TDL), serving as a default post-processing step before presenting the final hypothesis.

在这项工作中，我们引入了 CES——一个面向 OWL 类表达式的句法化简算法，并展示了它在降低冗长度和提升推理效率上的有效性。我们要强调的一点是：**该方法可以无缝集成进类表达式学习系统（例如与 TDL 配合），作为呈现最终假设之前的默认后处理步骤。**

Since the simplifications are purely syntactic and semantics-preserving, the procedure introduces no risk of information loss, while consistently offering potential benefits with minimal time overhead. The simplification time cost becomes even less significant when considering the fact that class expressions can be reused.

由于这些化简是纯句法的、且保持语义，该流程不带来信息丢失的风险，同时以极小的时间开销持续提供潜在收益。考虑到类表达式可以被复用这一事实，化简的时间成本就更微不足道了。

We also acknowledge certain limitations. First, all tested hypotheses were generated by TDL. Consequently, the reported reduction rates may not be representative of concepts produced by other concept learning systems, particularly those that generate shorter or structurally different expressions.

我们也承认存在一些局限。**第一，所有被测试的假设都由 TDL 生成。**因此，报告中的缩减率未必能代表其它概念学习系统产出的概念，尤其是那些生成更短或结构不同的表达式的系统。

Another limitation of CES is the order of applying specific processes (e.g., factorization, merging of cardinality restrictions). Since the simplification process proceeds in a single pass, the ordering can influence the outcome. Determining an optimal application strategy remains an open question, and exploring multi-pass or rule-prioritization schemes may yield further improvements.

CES 的另一个局限是特定处理步骤的应用顺序（例如因式分解、基数限制的合并）。由于化简过程是单遍进行的，顺序会影响结果。确定一个最优的应用策略仍是开放问题，探索多遍或规则优先级方案可能带来进一步的改进。

Despite these limitations, our results demonstrate that substantial reductions in class expression complexity can be achieved, improving reasoning efficiency and reducing verbosity. As such, CES provides a practical step towards more interpretable and maintainable ontology-based systems.

尽管有这些局限，我们的结果表明类表达式的复杂度可以获得实质性的降低，从而提升推理效率、减少冗长。因此，CES 为构建更可解释、更易维护的本体系统提供了实用的一步。

At present, CES operates exclusively at the syntactic level. This design choice avoids the need to rely on a reasoner, making the method broadly applicable and independent of the presence of a background ontology. Looking ahead, an exciting direction for future work is the incorporation of semantic simplifications, which could further reduce complexity and uncover additional equivalences beyond purely syntactic transformations.

目前 CES 只在句法层面工作。这个设计选择使它不必依赖推理机，从而具有广泛的适用性，并且不依赖于背景本体是否存在。展望未来，一个令人期待的方向是纳入**语义化简**——它可以进一步降低复杂度，发现那些超出纯句法变换范围的额外等价关系。

📌 相关链接

- [arXiv:2608.18899](#) — 本文原文 (cs.AI, 2026-08-19 提交)
- [OWLAPY](#) — Paderborn 大学 DICE 组的 Python OWL 本体工程框架，CES 实现在其中
- [Ontolearn](#) — 同一组的大规模 OWL 类表达式学习框架，产出本文所化简表达式的 TDL 学习器在其中
- [SML-Bench](#) — 结构化机器学习基准套件，本文用的 Carcinogenesis 与 Mutagenesis 两个数据集出自这里
- [FaCT++](#) — 经典 OWL DL 推理机，其简化范式 (SNF) 是本文最接近的对比点
- [OWL 2 Web Ontology Language](#) — W3C 的 OWL 2 标准总览，[SR0IQ\(D\)](#) 是 OWL 2 DL 对应的描述逻辑档位